Course Contents

Full Stack Python Course Contents

Audience: Freshers & career switchers | **Stack:** Python 3, MySQL 8+, Django 5.x, HTML/CSS/JS, React, GitHub, VS Code
Duration (default plan): 20 weeks × 1 hour 15 minutes/day (Mon–Fri live) = **100 sessions / ~125 live hours** + homework
Schedule authority: `PYTHON/Session_Calendar_100.md`
Practice DB: `training_db` | **IDE:** VS Code | **VCS:** Git + GitHub
Legend: Must = job-ready path | *Optional* = deepen if hours allow / advanced batch

Syllabus source: KSR Datavizon *FULL STACK PYTHON 2026–27* brochure (mandatory modules preserved; sequence improved for fresher learning).

> Live class length is **1h 15m**. One session = one focus. Required SQL labs live in repo `MYSQL/` (versioned external dependency).

Program map (improved sequence)

Phase	Weeks	Focus	Outcome
0–1	1–2	Computer + Core Python foundations	Think like a programmer; write small CLI scripts
2	3–6	Collections + functions start	Build reusable Python tools
3	7–9	OOP, files, logging, Git	Intermediate Python + GitHub habit
4	10–11	MySQL (**required labs**) + Python DB	CRUD apps talking to MySQL
5	12–13	HTML, CSS, JavaScript + Django start	Front pages + first Django site
6	14–16	Django auth, labs `06` – `07`, capstone shell	Auth CRUD + deploy checks
7	17–18	Flask lite + DRF + React	REST APIs & SPA integration
8	19–20	Testing, capstone, placement (**AI optional**)	Portfolio + interview readiness

1. Computer & Programming Foundations (Must)

- What is a computer, software, OS, files/folders
- What is programming? Algorithms vs code
- How websites and apps work (client–server, browser, server)

- Roles: Python developer, Django developer, full-stack developer (India market)
- Tools: VS Code, terminal/command prompt, browser DevTools overview

Practice: create folders, run terminal commands, open VS Code
Assignment: one-page “How a website works” flowchart

---

2. Core Python — Setup & Basics (Must)

- Introduction to Python & why industry uses it
- Install Python 3 + VS Code (+ optional PyCharm)
- Keywords, identifiers, variables, data types
- Operators
- Input and output statements
- Flow control: `if` / `elif` / `else`, `for`, `while`, `break` / `continue`

Practice: even/odd, grade calculator, multiplication table, number guessing
Assignments: Simple Calculator, ATM menu (CLI)

---

3. Core Python — Collections & Strings (Must)

- String (methods, slicing, formatting)
- List
- Tuple
- Set
- Dict
- Data structures overview (list vs stack/queue ideas for interviews)

Practice: student marks analysis, word frequency, contact book
Mini project: Student Marks Manager (CLI)

---

4. Advanced Python (Must)

- Functions; *decorators & generators*
- Modules & packages
- OOP: class, object, constructor, encapsulation, inheritance, polymorphism, abstract class / abstract method, interface (`ABC` / Protocol)
- Exception handling
- File handling
- Logging
- Database programming overview (bridge to MySQL/Python connector)

Mini project: Expense Tracker CLI with files + classes → push to GitHub

---

5. Git & GitHub (Must)

- `git init`, status, add, commit, log, branch, merge
- GitHub: repo, clone, push, pull, README, `.gitignore`
- Collaboration basics (PR overview)

Practice: push every mini project

---

6. Database (MySQL) (Must)

- Full-stack bridge + **required MySQL labs table**: `PYTHON/05_Database_MySQL/01_MySQL_for_Full_Stack_Developers.md` (points into `MYSQL/` — mandatory for placement)
- DDL, DML, DQL, TCL, JOINS, normalization overview
- Python–MySQL connectivity (Advanced Python bridge module)

Practice: employee / student schema
Assignment: Library or E-commerce schema + CRUD queries
Detail: `MYSQL/Course_Contents.md` / `MYSQL/` modules

---

7. Front-End Fundamentals (Must)

- **HTML:** structure, forms, tables, semantic tags
- **CSS:** selectors, box model, Flexbox, responsive design
- **JavaScript:** variables, functions, DOM, events, forms, fetch basics

Mini project: Responsive landing page + contact form validation

---

8. Django (Must)

- Introduction to Django — `07_Django/01_…`
- URLs, views, templates, static — `02_…`
- Models, ORM, admin — `03_…`
- Forms, sessions, authentication — `04_…`
- Security & deployment (`load_dotenv`, `STORAGES`, `check --deploy`) — `05_…`
- **Protected lab:** CRUD + auth consolidation — `06_…`
- **Protected lab:** deploy + capstone debug — `07_…`

Capstone seed: Blog / Task Manager / Booking System / CampusCart

> Django package = **seven files** (`01` – `07`). There are no modules 08–12.

---

9. Flask (*Must overview*)

- Flask vs Django (when to use which)
- Minimal routes, templates, JSON responses
- Small API or micro-service demo

---

10. APIs, JSON, Postman (Must)

- What is an API? REST principles
- JSON read/write in Python
- Postman collections: GET/POST/PUT/DELETE

- Consume third-party APIs; integrate front end ↔ Django/DRF

---

11. React (Must)

- Introduction, install & setup (Vite)
- Components, Props, State
- Component lifecycle / effects overview
- Event handling, Lists & Keys, Forms
- Styling, Routing
- Hooks (`useState`, `useEffect`)
- API integration; connecting frontend to Django backend
- Unit testing overview
- Project build and deployment

---

12. Testing, Debugging, Clean Code (Must)

- Debugging in VS Code
- Logging review
- Unit testing with `unittest` / `pytest` basics
- Clean-code habits for juniors

---

13. AI Integration (*Optional stretch / brochure*)

- **Only after** deployed Django MVP + SQL confidence + GitHub portfolio
- AI Integration with Python programming (overview)
- Calling ML models via APIs (scikit-learn demo or hosted API)
- Building simple AI-driven UI (chatbot-style form or text analyzer)
- Never blocks job-ready / placement gate

---

14. Miscellaneous — Soft Skills & Placement (Must)

- Agile (Scrum) overview
- Daily assignments & weekly assessments
- Resume and LinkedIn optimization
- GitHub portfolio
- Mock interviews, aptitude/coding prep
- Live / capstone project
- Job assistance process (India entry-level roles)

Suggested weekly load

Mode	Weeks	Live hours/day	Sessions	Approx. live total
Default	**20**	**1h 15m**	**100**	**~125h** (+ homework)
Comfortable	22–24	1h 15m	110–120	Extra Django/React buffer

Mode	Weeks	Live hours/day	Sessions	Approx. live total
Compressed	16	1h 15m + Sat lab	80+	Only with weekend labs

Adjust weeks only via the Session Calendar; keep module order fixed. **Do not plan as 4-hour daily classes.**